CloudLog Analytics: Cloud-Based Web Log Analysis Using MapReduce

1. PROJECT TITLE

CloudLog Analytics: Cloud-Based Web Log Analysis Using MapReduce

2. STUDENT NAME(S)

Student 1: [Student Name 1 — Roll Number / ID] | Student 2: [Student Name 2 — Roll Number / ID] | Student 3: [Student Name 3 — Roll Number / ID] (Optional)

3. OBJECTIVE

To implement an educational web log analytics platform applying the MapReduce paradigm to HTTP access logs, evaluate serial versus two-process execution performance on a single host, and prepare the application for managed hosting on Streamlit Community Cloud.

4. PROBLEM STATEMENT

Server access logs are vital for monitoring web service availability and diagnosing HTTP error spikes. Commercial cloud instances like Amazon EC2 require payment credentials and incur ongoing usage costs, while distributed cluster frameworks like Apache Hadoop introduce substantial multi-node hardware provisioning and Linux administration overhead. The goal is to build an authentic single-host MapReduce engine with rigorous schema validation and interactive visualizations without misrepresenting local multiprocessing as a distributed cluster.

5. METHODOLOGY / TOOLS USED

Python 3.11 in an isolated virtual environment (`.venv`). Educational single-host MapReduce pipeline (Partition, Map, Shuffle, Reduce, Derive) computing exact weighted means. Concurrency via serial baseline and dual-worker `ProcessPoolExecutor` with transparent fallback. Presentation via Streamlit web app with Plotly charts. Automated verification via 31 Pytest unit and integration tests. Deployment via free-tier Streamlit Community Cloud (managed PaaS).

6. IMPLEMENTATION

The system ingests web log CSV files under a 5.0 MB and 50,000-row limit, validating timestamps, methods, integer HTTP status codes (rejecting fractional values), and non-negative latencies. Validated records are partitioned across workers, mapped into key-value tuples, shuffled into grouped lists, reduced into cumulative sums, and derived into exact weighted means (`total_latency / total_requests`). The benchmark engine compares serial and two-process runs across top-level metrics and all three aggregate tables (endpoint, status code, hourly), verifying exact grouping keys, integer counts, and floating-point aggregates within documented tolerance (0.01 ms for latency, 0.01 percentage points for percentage metrics).

7. RESULTS

All 31 Pytest tests pass with zero warnings, verifying equivalence against independent Pandas aggregations. Mapper workers execute in child processes (`worker_pid != parent_pid`); forced worker pool failure falls back cleanly to serial execution with full output correctness. Serial and parallel runs match on all grouping keys, integer counts, and float metrics within tolerance (0.01 ms latency, 0.01 percentage points for percentage metrics). On this host machine, observed serial execution on 5,000 records (~15–30 ms) was faster than two-process mode (~1.5–2.8 s on Windows) due to process spawn and IPC overhead; these timings are host-specific observations rather than universal guarantees. Streamlit Community Cloud deployment remains pending final repository push and activation.

8. CONCLUSION

CloudLog Analytics proves that foundational cloud processing concepts can be demonstrated effectively using free, open-source tools. The project demonstrates the mathematical necessity of weighted derivations and provides an honest empirical look at multiprocessing overhead on modest workloads.

9. REFERENCES

1. Dean, J., & Ghemawat, S. (2004). MapReduce: Simplified Data Processing on Large Clusters. In *Proceedings of the 6th USENIX Symposium on Operating System Design and Implementation (OSDI '04)*, pp. 137–150.

2. Python Software Foundation. (2024). *concurrent.futures — Launching parallel tasks*. Python 3.11 Documentation.

3. Streamlit Inc. (2024). *Streamlit Community Cloud Documentation*. Snowflake Inc.